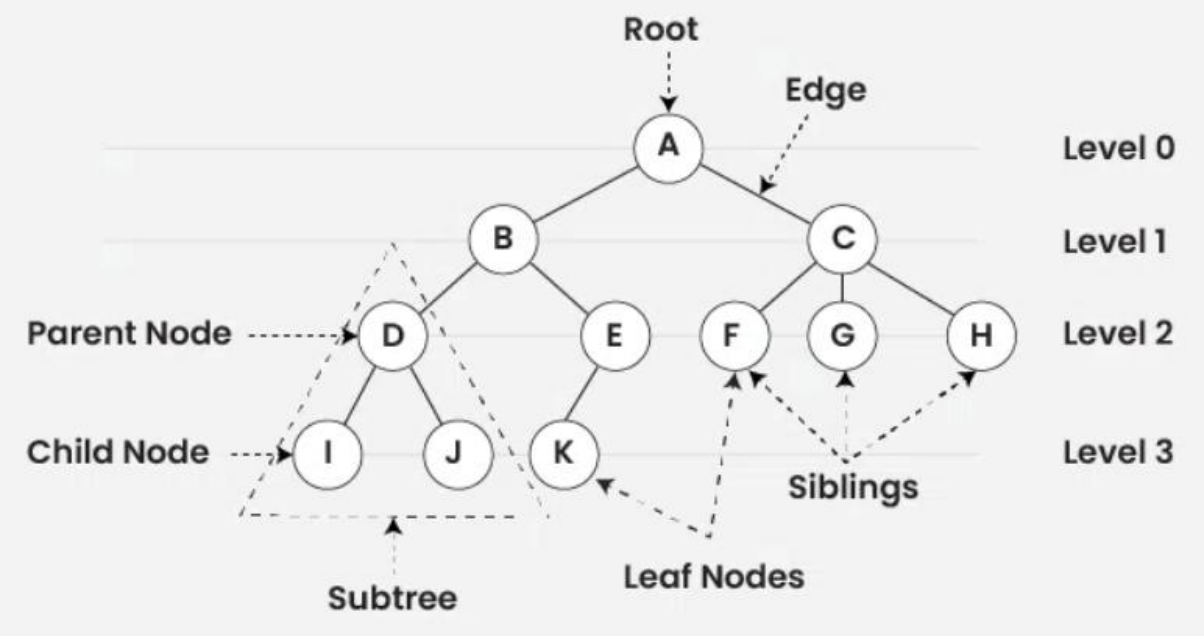
Tree:

1.Tree data structure is a hierarchical structure that is used to represent and organize data in the form of parent child relationship. Ex: Folder structure in an operating system.

2.The topmost node of the tree is called the **root**, and the nodes below it are called the child nodes. Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.



3.A tree consists of a root node, and zero or more subtrees T1, T2, ... , Tk such that there is an edge from the root node of the tree to the root node of each subtree.

4.The data in a tree are not stored in a sequential manner i.e., they are not stored linearly. Instead, they are arranged on multiple levels or we can say it is a hierarchical structure. For this reason, the tree is considered to be a non-linear data structure.

Basic/Important Terminologies in Tree Data Structure:

1.Root Node: The topmost node of a tree or the node which does not have any parent node is called the root node. **{A}** is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.

2.Parent Node: The node which is an immediate predecessor of a node is called the parent node of that node. **{B}** is the parent node of **{D, E}**.

3.Child Node: The node which is the immediate successor of a node is called the child node of that node. **Examples: {D, E} are the child nodes of {B}.**

4.Sibling: Children of the same parent node are called siblings. **{D, E}** are called siblings.

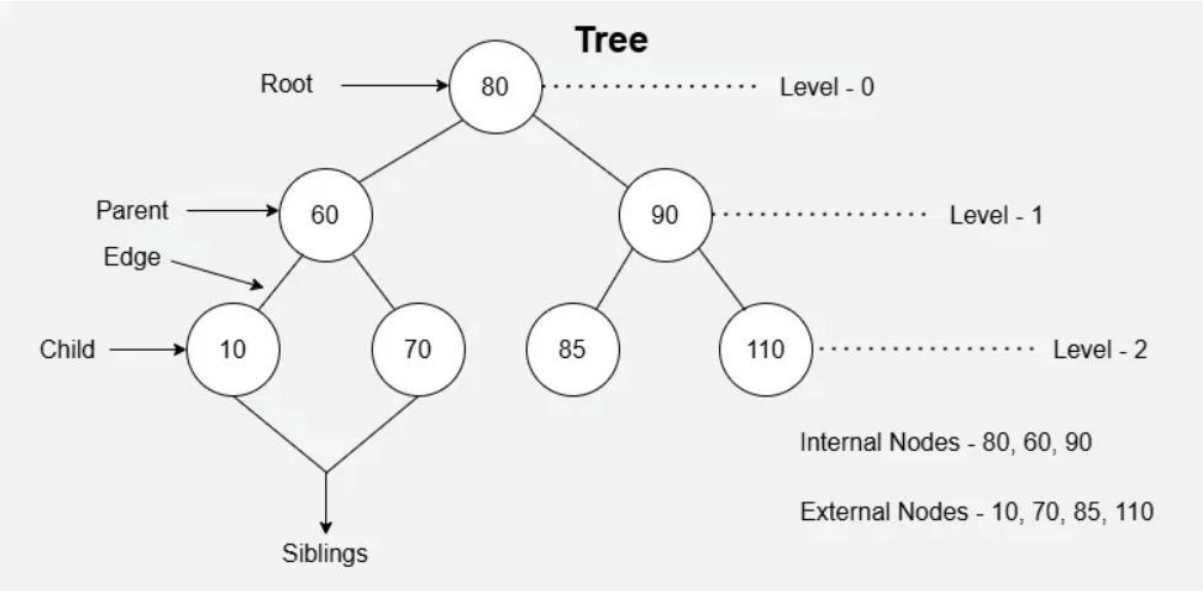
5.Leaf Node or External Node: The nodes which do not have any child nodes are called leaf nodes. **{I, J, K, F, G, H}** are the leaf nodes of the tree.

6.Path: Path is the number of successive edges from source node to destination node.

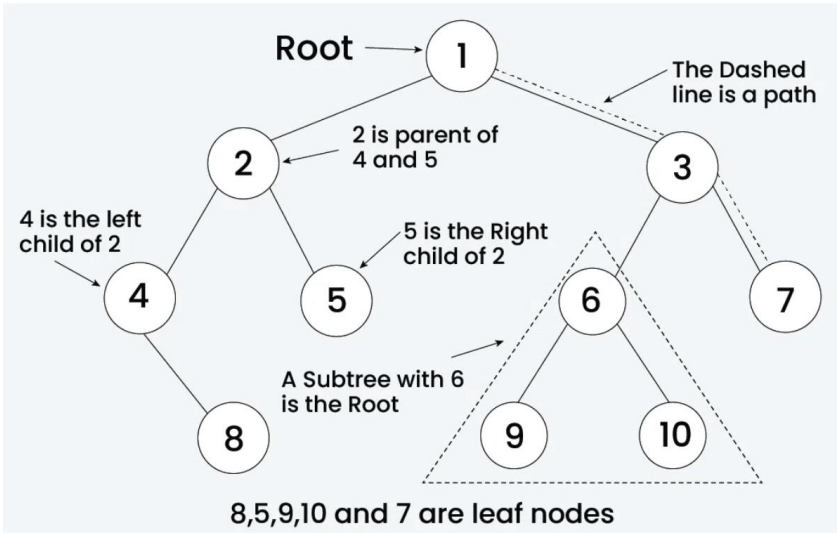
7.Degree of Node: Degree of a node in a tree refers to the number of direct children that node possesses. **NOTE:** When the degree of node is zero than that particular node is called leaf node.

8.Level: The levels of a tree are the horizontal layers of nodes. The root node, which is the topmost node, is considered to be at level 0. The children of the root node are at level 1, their children are at level 2, and so on.

9.Height of the tree or node: The height of a tree is defined as the length of the longest path from the root node to any leaf node. This length is measured by the number of edges along that path.



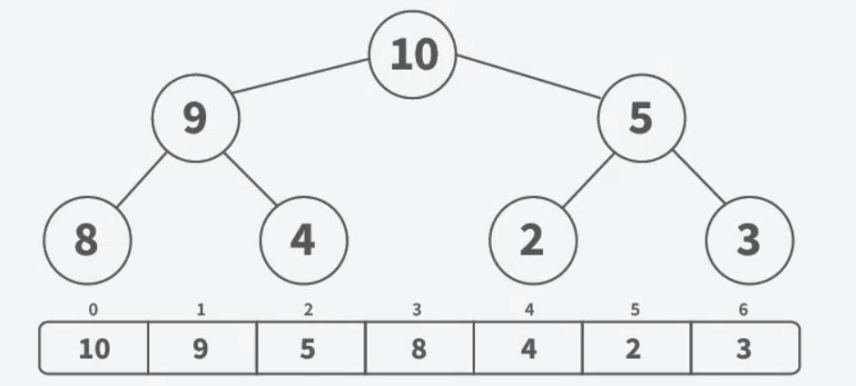
Binary Tree: A Binary Tree Data Structure is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. It is commonly used in computer science for efficient storage and retrieval of data, with various operations such as insertion, deletion, and traversal. The child nodes of the binary tree are ordered.



Representation of Binary Tree:

1.Array Representation: Array Representation is a way to represent binary trees, especially useful when the tree is complete (all levels are fully filled except possibly the last, which is filled from left to right). **In this method:**

- The tree is stored in an array.
- For any node at index **i**:
  - Left Child**: Located at 2 * i + 1**
  - Right Child: **Located at 2 * i + 2**
- Root Node: Stored at index 0.



Advantages:

1.Easy to navigate parent and child nodes using index calculations, which is fast

2.Easier to implement, especially for complete binary trees.

Disadvantages:

1.You have to set a size in advance, which can lead to wasted space.

2.If the tree is not complete binary tree then then many slots in the array might be empty, this will result in wasting memory

3.Not as flexible as linked representations for dynamic trees.

NOTE:

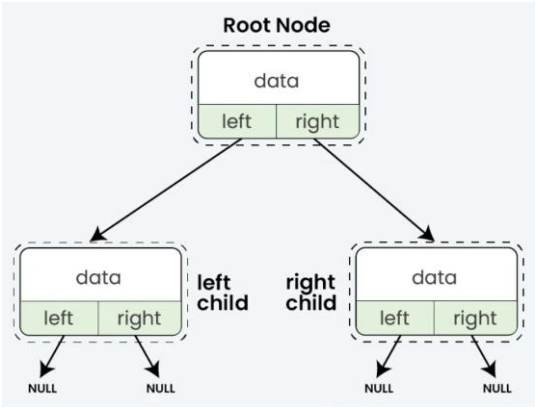
1.The nodes are numbered sequentially level by level from left to right, where empty nodes are also numbered.

2.When the data of the tree is represented in an array, the number appearing against the node is the subscript number for array.

3.The location index 0 is used to store the size of tree in the number of nodes both existing and non-existing.

4.In the array, all the non-existing node is represented using null character "\θ".

2.Linked List Representation (Dynamic): This is the simplest way to represent a binary tree. Each node contains data and pointers to its left and right children.



Advantages:

1.It can easily grow or shrink as needed, so it uses only the memory it needs.

2.Adding or removing nodes is straightforward and requires only pointer adjustments.

3.Only uses memory for the nodes that exist, making it efficient for sparse trees.

Disadvantages:

1.Needs extra memory for pointers.

2.Finding a node can take longer because you have to start from the root and follow pointers.

Syntax to declare a node of "linked list representation of binary tree":

1.Using struct:

```
STRUCT NODE {
  INT DATA;
  NODE* LEFT, * RIGHT;

  NODE(INT VAL) {
    DATA = VAL;
    LEFT = NULLPTR;
    RIGHT = NULLPTR;
  }
};
```

2.Using class:

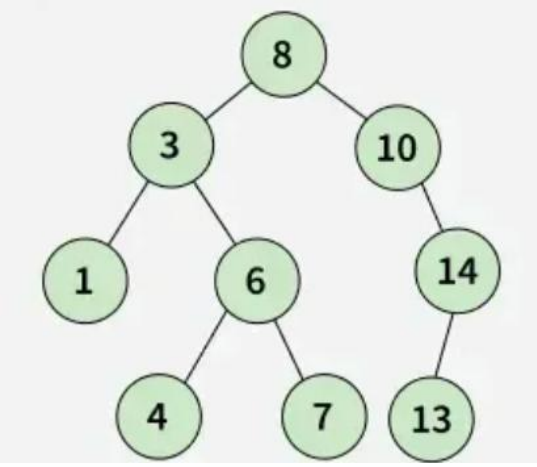
```
CLASS NODE {
PUBLIC:
  INT DATA;
  NODE* LEFT, * RIGHT;

  NODE(INT VAL) {
    DATA = VAL;
    LEFT = NULLPTR;
    RIGHT = NULLPTR;
  }
};
```

Binary Search Tree: A Binary Search Tree (BST) is a type of binary tree data structure in which each node contains a unique key and satisfies a specific ordering property:

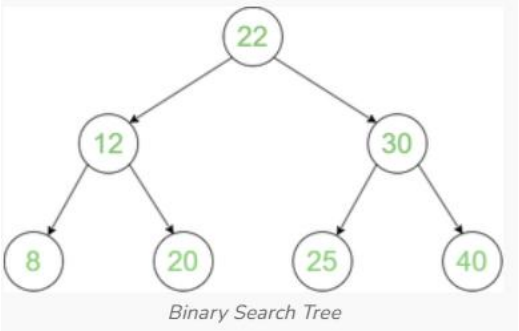
i. All nodes in the left subtree of a node contain values strictly less than the node's value.

ii. All nodes in the right subtree of a node contain values strictly greater than the node's value.

This structure enables efficient operations for searching, insertion, and deletion of elements, especially when the tree remains balanced.



Basic Operations:
1.Insertion
2.Searching
3.Deletion
4.Traversal
Ways of Traversing in BST:



Binary Search Tree

1.Inorder Traversal:
i.Traverse left subtree.
ii.Visit the root.
iii.Traverse the right subtree.

Inorder Traversal: 8 12 20 22 25 30 40

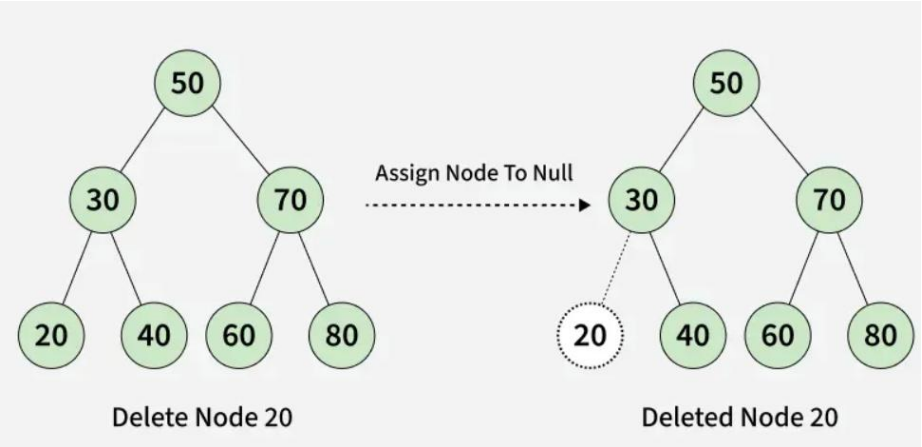
2.Preorder Traversal:
i.Visit the root.
ii.Traverse left subtree.
iii.Traverse the right subtree.

Preorder Traversal: 22 12 8 20 30 25 40

3.Postorder Traversal:
i.Traverse left subtree.
ii.Traverse the right subtree.
iii.Visit the root.

Postorder Traversal: 8 20 12 25 40 30 22

Deletion in BST: Given a BST, the task is to delete a node in this BST, which can be broken down into 3 scenarios:
Case 1: Node to be deleted is a leaf node.
Solution: Just remove the node.



Case 2: Node to be deleted has one child node.
Solution: Remove the node and connect its parent directly to its child.

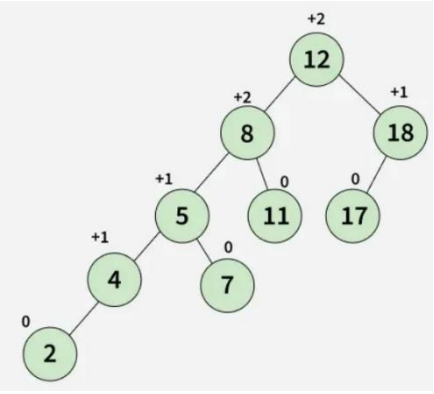
Case 3: Node to be deleted has 2 child nodes.
Solution: There are two way to delete the node:
i.Replace node to be deleted by its inorder successor.
ii.Replace node to be deleted by its inorder predecessor.

AVL (Adelson-Velsky-Landis) Tree: An AVL tree is a self-balancing binary search tree in which the height difference between the left and right subtrees of every node is at most one, maintained through rotations after insertions and deletions.

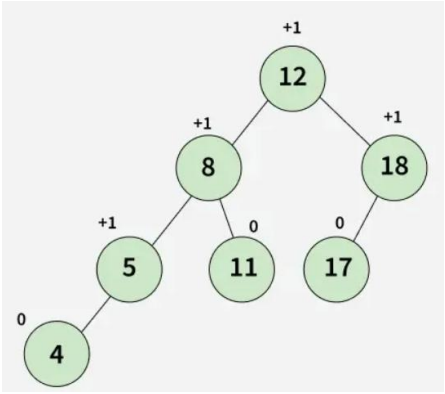
1.In other words, for every node in an AVL tree:

- **Balance Factor = Height(left subtree) − Height(right subtree) ∈ {−1,0,+1}**

2.This property ensures that the tree remains balanced, preventing it from becoming skewed like a normal binary search tree.
3.As a result, operations such as searching, insertion, and deletion can be performed efficiently in O(log n) time.
4.Whenever an insertion or deletion operation causes the tree to become unbalanced, the AVL tree automatically performs rotations (single or double) to restore its balance.



Example of a BST which is not an AVL Tree

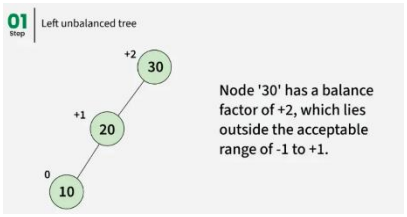


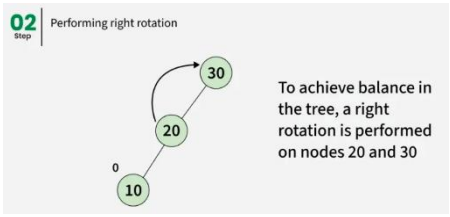
Example of an AVL Tree

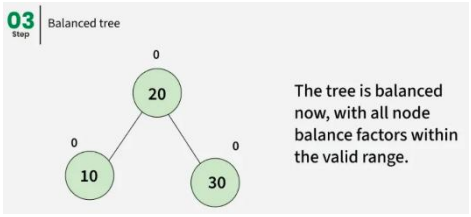
Rotation in AVL tree: Rotations are designed to restore balance in O(1) time while ensuring the overall time complexity remains O(log n). AVL Trees use four types of rotations to rebalance themselves after insertions and deletions: Left-Left (LL) Rotation, Right-Right (RR) Rotation, Left-Right (LR) Rotation, Right-Left (RL) Rotation.

1.Left-Left Rotation:

- Occurs when a node is inserted into the left subtree of the left child, causing the balance factor to become more than +1.
- Fix: Perform a single right rotation.

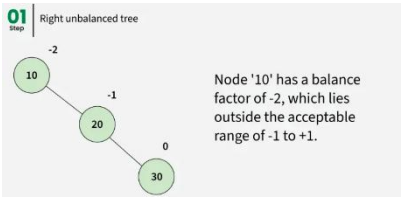


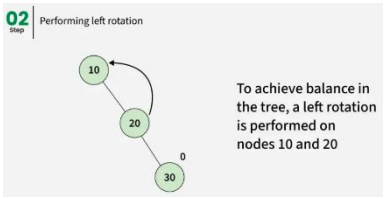


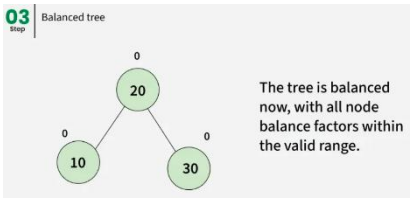


2.Right-Right Rotation:

- Occurs when a node is inserted into the right subtree of the right child, making the balance factor less than -1.
- Fix: Perform a single left rotation.

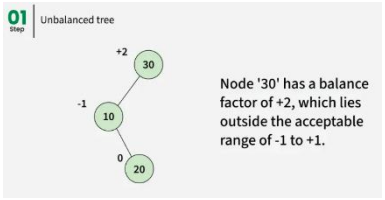


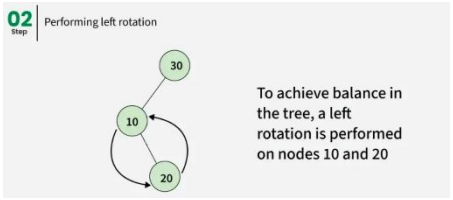


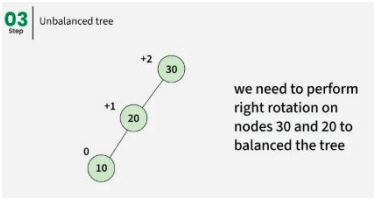


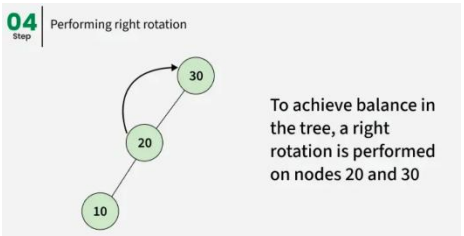
3.Left-Right Rotation:

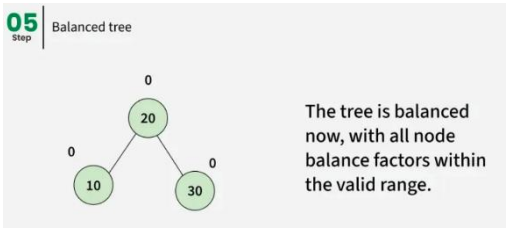
- Occurs when a node is inserted into the right subtree of the left child, which disturbs the balance factor of an ancestor node, making it left-heavy.
- Fix: Perform a left rotation on the left child, followed by a right rotation on the node.





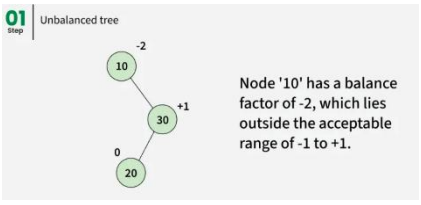


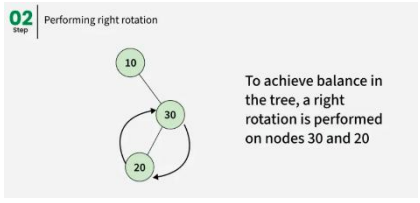


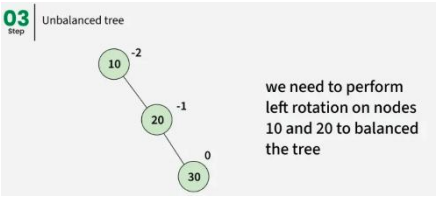


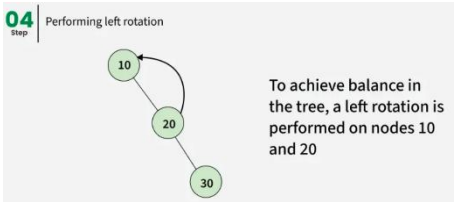
4.Right-Left Rotation:

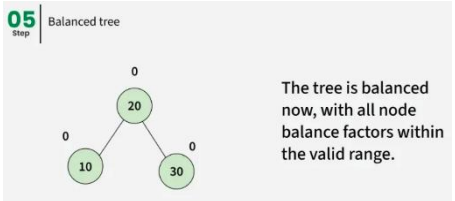
- Occurs when a node is inserted into the left subtree of the right child, which disturbs the balance factor of an ancestor node, making it right-heavy.
- Fix: Perform a right rotation on the right child, followed by a left rotation on the node.







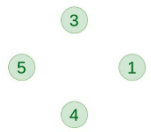




NOTE:
1.Left Heavy → More nodes (or taller height) on the **left side**.
2.Right Heavy → More nodes (or taller height) on the **right side**.
3.When the absolute value of the balance factor becomes greater than 1 (|BF| > 1), the AVL tree becomes unbalanced, and rotations are required to fix it.
4. Every AVL Tree is a BST, but NOT every BST is an AVL Tree.
Because AVL tree follows all BST rules *plus* an extra balance condition to keep the height minimal.

Graph: Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(V) and a set of edges(E). The graph is denoted by G(V, E).

Types of Graph:
1.Null Graph: A graph is known as a null graph if there are no edges in the graph.

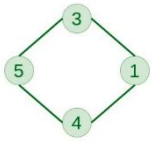


Null Graph

2.Trivial Graph: Graph having only a single vertex, it is also the smallest graph possible.

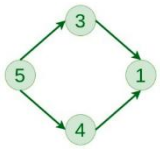
Trivial Graph

3.Undirected Graph: A graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.



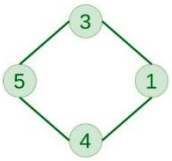
Undirected Graph

4.Directed Graph: A graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.



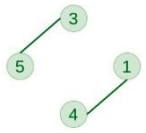
Directed Graph

5.Connected Graph: The graph in which from one node we can visit any other node in the graph is known as a connected graph.



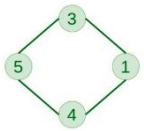
Connected Graph

6 Disconnected Graph: The graph in which at least one node is not reachable from a node is known as a disconnected graph.



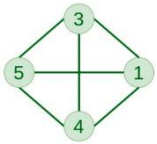
Disconnected Graph

7.Regular Graph: The graph in which the degree of every vertex is equal to K is called K regular graph.



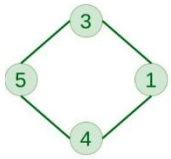
2-Regular

8.Complete Graph: The graph in which from each node there is an edge to each other node.



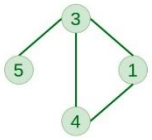
Complete Graph

9.Cycle Graph: The graph in which the graph is a cycle in itself, the minimum value of degree of each vertex is 2.



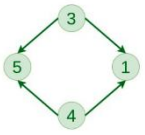
Cycle Graph

10.Cyclic Graph: A graph containing at least one cycle is known as a Cyclic graph.



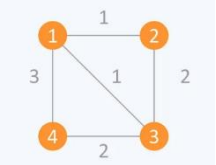
Cyclic Graph

11.Directed Acyclic Graph: A Directed Graph that does not contain any cycle.



Directed Acyclic Graph

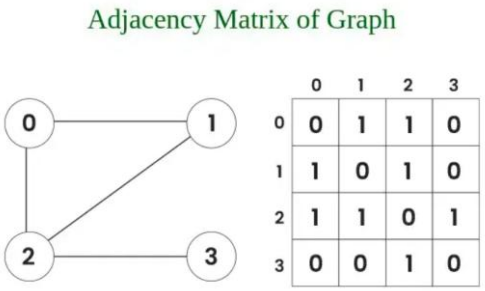
13.Weighted Graph: A graph in which the edges are already specified with suitable weight is known as a weighted graph. Weighted graphs can be further classified as directed weighted graphs and undirected weighted graphs.



Weighted Graph

Representation of Graph: There are multiple ways to store a graph: The following are the most common representations: **i. Adjacency Matrix, ii. Adjacency List.**

1.Adjacency Matrix: An Adjacency Matrix is a 2D array used to represent a graph, where each cell (i, j) indicates whether there is an edge between vertex i and vertex j.



Steps to Create an Adjacency Matrix:

1.Number all vertices from 0 to n-1 (or 1 to n).

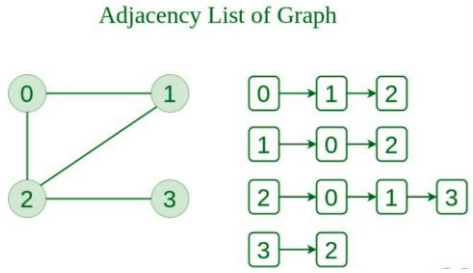
2.Create a 2D array adj[n][n] and initialize all elements to 0.

3.For every edge (u, v) in the graph:

- **If the graph is undirected:**
  - adj[u][v] = 1
  - adj[v][u] = 1
- **If the graph is directed:**
  - adj[u][v] = 1

4.If the graph is weighted, store the weight instead of 1.

2.Adjacency List: An Adjacency List is a collection of lists or vectors, where each vertex has a list of all the vertices directly connected to it.



Steps to Create an Adjacency List:

1.Create an array (or vector) of size n for all vertices.

2.For every edge (u, v) in the graph:

- Add v to the list of u.
- If the graph is undirected, also add u to the list of v.

3.If the graph is weighted, store pairs (v, weight) instead of only v.

Degree of a node: The degree of a node (or vertex) in a graph is the number of edges that are connected to that vertex.

Types of Degree:

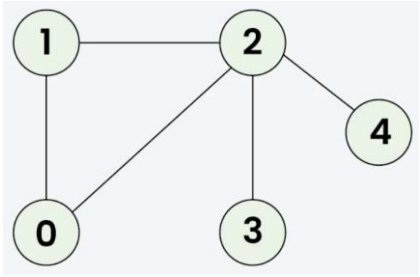
1.Indegree: The indegree of a vertex is the number of edges coming *into* that vertex. In other words, it counts how many vertices point toward that vertex.

2.Outdegree: The outdegree of a vertex is the number of edges going *out from* that vertex. It counts how many vertices that vertex points to.

Graph traversals: Graph traversal, also known as graph search, is the process of systematically visiting all the nodes (vertices) and edges of a graph data structure. The primary goal is to explore the entire graph or a specific part of it, which is essential for various applications.

Types of Traversals:

1.Depth-First Search: In Depth First Search (or DFS) for a graph, we traverse all adjacent vertices one by one. When we traverse an adjacent vertex, we completely finish the traversal of all vertices reachable through that adjacent vertex.



Note : There can be multiple DFS traversals of a graph according to the order in which we pick adjacent vertices. Here we pick vertices as per the insertion order.

Output: *[0 1 2 3 4]*

Explanation: *The source vertex s is 0. We visit it first, then we visit an adjacent.*

Start at 0: Mark as visited. Output: 0

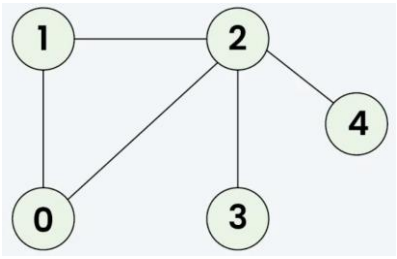
Move to 1: Mark as visited. Output: 1

Move to 2: Mark as visited. Output: 2

Move to 3: Mark as visited. Output: 3 (backtrack to 2)

Move to 4: Mark as visited. Output: 4 (backtrack to 2, then backtrack to 1, then to 0)

2.Breadth-First Search: Breadth First Search (BFS) is a graph traversal algorithm used to explore all the vertices and edges of a graph in a breadthward (level-by-level) manner, starting from a given source vertex. It visits all the vertices at the current level before moving to the next level of vertices. BFS uses a queue data structure to keep track of vertices that need to be visited next.



Output: *[0, 1, 2, 3, 4]*

Explanation: *Starting from 0, the BFS traversal proceeds as follows:*

Visit 0 → Output: [0]

Visit 1 (the first neighbor of 0) → Output: [0, 1]

Visit 2 (the next neighbor of 0) → Output: [0, 1, 2]

Visit 3 (the first neighbor of 2 that hasn't been visited yet) → Output: [0, 1, 2, 3]

Visit 4 (the next neighbor of 2) → Final Output: [0, 1, 2, 3, 4]

AOV / Activity-On-Vertex Network: Activity-on-Vertex (AOV) network is a directed graph where vertices represent tasks and edges represent precedence relations, indicating that one task must be completed before another can begin.

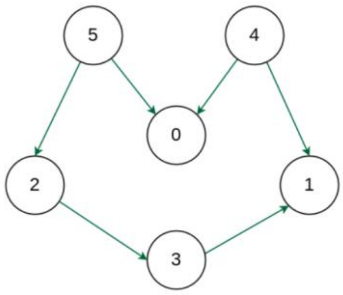
Key Characteristics:
1.Directed Graph: The connections between tasks have a specific direction, indicating the flow from one task to the next.
2.Vertices as Tasks: Each node (vertex) in the graph represents a specific activity or step in a project.
3.Edges as Precedence: The arrows (edges) between vertices show that a task must be completed before the task it points to can start.
4.No Cycles: A valid AOV network must be a Directed Acyclic Graph (DAG), meaning it should not contain any cycles, as a cycle would imply a task depends on itself, making it impossible to complete.

Used in: PERT (Program Evaluation and Review Technique) or CPM (Critical Path Method).
It helps to identifies:
1.Bottlenecks: Activities with many dependent tasks can be identified as bottlenecks, which may need more attention or resources.
2.Dependencies: Shows which activities depend on others. Helps avoid starting a task before its preceding tasks are finished.
3.Project Duration: By analyzing all paths from the start to end vertex, we can estimate the minimum time required to complete the project.

Purpose and Application:
1.Project Scheduling: AOV networks help in planning and managing projects by visualizing the dependencies between tasks.
2.Topological Sort: The primary use of an AOV network is to perform a topological sort. This process finds a linear ordering of the vertices such that for every directed edge from vertex u to vertex v, u comes before v in the ordering. This order dictates the sequence in which tasks should be performed.
3.Feasibility Assessment: By performing a topological sort, you can verify if a project is feasible and identify any potential conflicts or circular dependencies.

Topological Sorting:
1.Topological sorting is a linear ordering of vertices in a Directed Acyclic Graph (DAG) where for every directed edge from vertex U to vertex V, U always comes before V in the ordering.
2.It's an algorithm used to represent and linearize tasks with dependencies, such as project management, software compilation, and dependency resolution.
3.A topological sort only exists for DAGs, as a cycle would mean two vertices depend on each other, making a linear ordering impossible.
4.Can have multiple topological ordering.

Kahn's algorithm for Topological Sorting:



Input: V = 6, edges = [[2, 3], [3, 1], [4, 0], [4, 1], [5, 0], [5, 2]]

Vertices->	0	1	2	3	4	5
Indegree	2	2	1	1	0	0
	1	1	1	1	X	0
	0	1	0	1	X	X
	X	1	0	1	X	X
	X	1	X	0	X	X
	X	0	X	X	X	X
	X	X	X	X	X	X

Topological Order: 4, 5, 0, 2, 3, 1

Working of the Algorithm:

1.Add all nodes with in-degree 0 to a queue.

2.While the queue is not empty:

- Remove a node from the queue.
- For each outgoing edge from the removed node, decrement the in-degree of the destination node by 1.
- If the in-degree of a destination node becomes 0, add it to the queue.

3.If the queue is empty and there are still nodes in the graph, the graph contains a cycle and cannot be topologically sorted.

4.The nodes in the queue represent the topological ordering of the graph.

AOE / Activity-On-Edge Network: An AOE network is a directed graph used in project management to represent a project's activities and events - mainly in Critical Path Method (CPM) analysis.

An AOE network is also a directed graph used in project scheduling (PERT/CPM) where:

1.Edges (arrow) represent activities of the project.

2.Vertices (nodes) represents events (the start or completion of one or more activities).

3.Also called AOA (activity-On-Arrow).

Features:

1.Edges are Activities: Each directed edge in the graph symbolizes a specific task or activity that needs to be performed**.**

2.Vertices are Events: Vertices, or nodes, represent events that signify the completion of one or more activities. An event can only occur once all incoming activities have been finished.

3.DAG No Cycles: The AOE network is a Directed Acyclic Graph (DAG); meaning it has no loops, as time cannot flow backward.

4.Dummy Activities: Sometimes we add a dummy edge (0 duration) to represent precedence relations when no real activity exists.

Applications:

1.Project Scheduling: Helps arrange activities in proper sequence. Determines start, finish, and total project duration.

2.Critical Path Method (CPM): Identifies the critical path and zero slack activities. Assists in estimating and controlling project duration.

3.PERT (Program Evaluation and Review Technique): Used for probabilistic time estimation. Helps calculate expected completion time and deadline probability.

4.Time Analysis: Calculates ES, EF, LS, LF, and Slack for each activity. Aids in analyzing and optimizing project time.

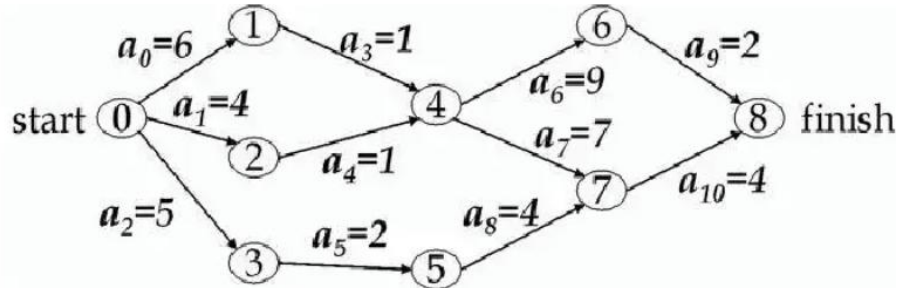
5.Resource Utilization: Improves resource allocation and avoids conflicts.
Ensures efficient use of manpower, machines, and materials.

NOTE:

1.Choose large early finish for forward finish.

2.Choose small late start for backward pass.

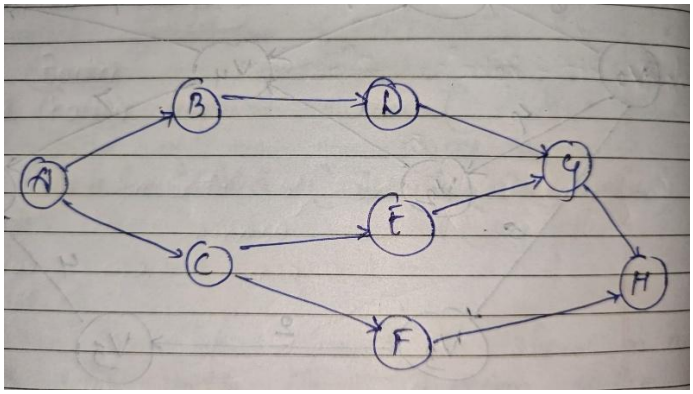
3.Late Start and Early Start OR Late Finish and Early Finish should be equal for the critical path.

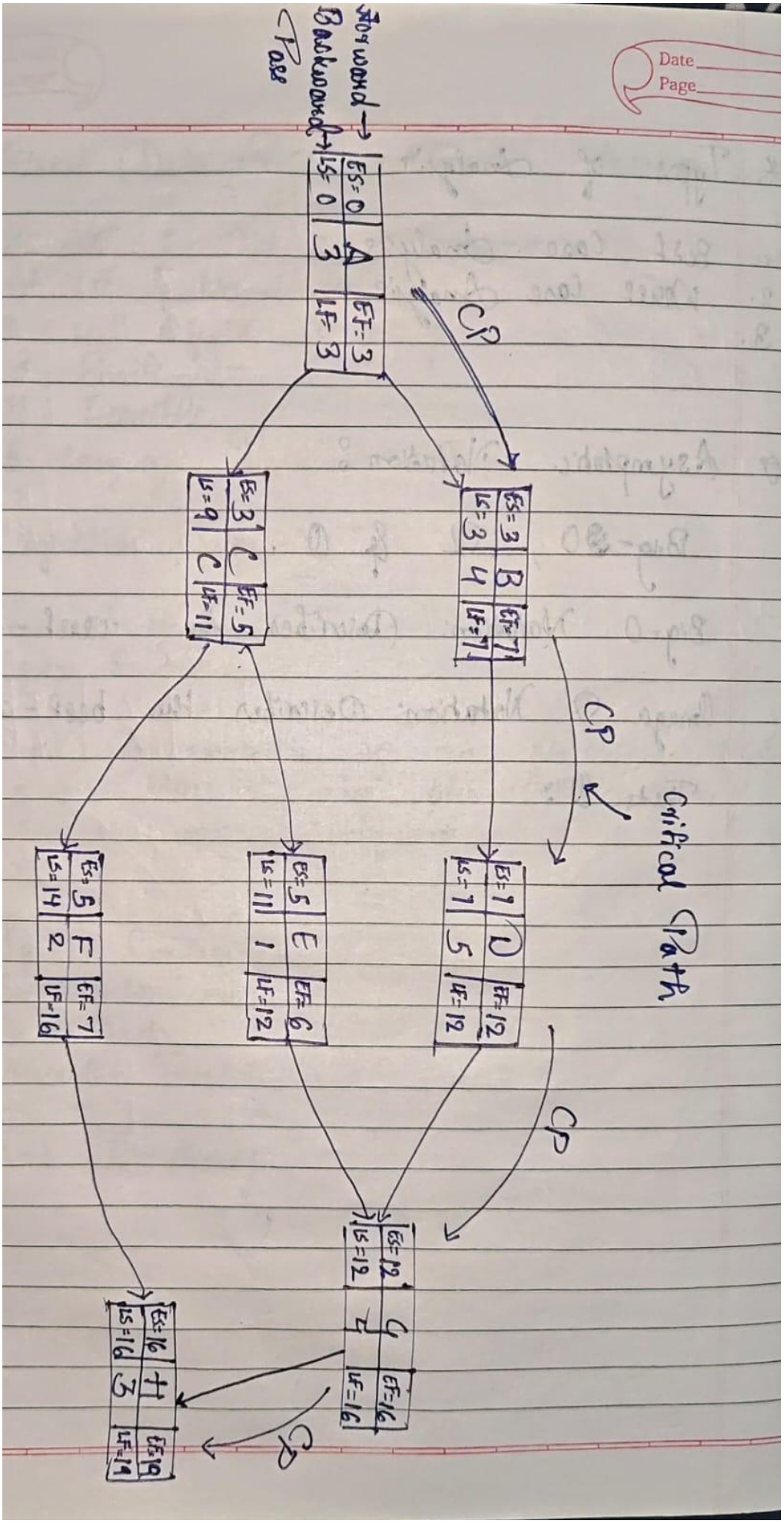


1.Start Event: V0 (No incoming arrows – 0 indegree).

2.Sink Node: V8 (No outgoing arrows – 0 outdegree).

Activity	Predecessor	Duration
A	-	3
B	A	4
C	A	2
D	B	5
E	C	1
F	C	2
G	D, E	4
H	F, G	3




Forward Pass
Backward Pass
ES=0
LS=0
A
3
EF=3
LF=3
ES=3
LS=3
B
4
EF=7
LF=7
ES=3
LS=9
C
C
EF=5
LF=11
ES=1
LS=7
D
5
EF=12
LF=12
ES=5
LS=14
F
2
EF=7
LF=16
ES=5
LS=11
E
1
EF=6
LF=12
ES=12
LS=12
G
4
EF=16
LF=16
ES=16
LS=16
H
3
EF=19
LF=19
CP
CP
CP
CP
Critical Path